B11 Machine learning workflow

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B11 Machine learning workflow 3
B11-L1 Train, validation and test, split by the right unit 3
B11-L2 Pipeline and ColumnTransformer 6
B11-L3 Cross-validation and honest hyperparameter search 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B11 Machine learning workflow

Roadmap: Prerequisite for all months. Splits, Pipeline, ColumnTransformer, cross-validation and hyperparameter search.

B11-L1 Train, validation and test, split by the right unit

Concept

Three sets, three jobs:

- **Training** rows fit the model.
- **Validation** rows, or cross-validation folds within training, choose between models and settings.
- **Test** rows measure the final choice, **once**. Every extra look at the test set turns it into a second validation set, and its score stops being an honest estimate.

Split so the test set looks like the data the model will meet.

Stratify on the target when a class is rare. `train_test_split(X, y, stratify=y)` keeps a 3% positive rate at 3% in both sets. Without it, a small test set can end up with very few positives.

Group when rows are not independent. A panel respondent answering five waves appears in five rows. A random row split puts some of their waves in training and others in test. The model then recognises the **person** instead of learning the pattern, and the test score measures memory.

A demonstration from this course's build (scikit-learn 1.8, random forest): 200 simulated respondents, 5 waves each, a label that depends only on the person and is unrelated to the features. Random 5-fold CV reported accuracy **0.995**. `GroupKFold` by respondent reported **0.468**: chance, which is the truth.

```
from sklearn.model_selection import GroupKFold, GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=7)
train_idx, test_idx = next(splitter.split(df, groups=df["respondent_id"]))
```

The group is whatever repeats: respondent, household, store, patient, device. If it will be **new** in production, keep it out of training in validation too. For time-ordered use, split by time as well (lesson B10-L3).

Record the split: the seed, the group column, and the resulting row counts, in the run manifest (lesson B6-L2).

Why it matters

A model predicting survey drop-out was validated with a random row split over a panel dataset. It scored AUC 0.93. Deployed on a newly recruited panel, where every respondent was new, it scored 0.61. Each respondent's answering style had let the model match test rows to the same person's training rows. A group split would have shown 0.61 before deployment.

Worked example: weak code

```
train, test = train_test_split(panel, test_size=0.2)
for params in grid:
    score = evaluate(fit(train, params), test)
```

1. **Line 1.** Random rows: the same respondent sits on both sides.
2. **Line 3.** Tuning on the test set: its score is no longer an estimate of anything.

Worked example: strong code

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit


def split_by_group(
    df: pd.DataFrame, group_col: str, test_size: float, seed: int
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Split so that no group appears in both train and test."""
    splitter = GroupShuffleSplit(n_splits=1, test_size=test_size,
    random_state=seed)
    train_idx, test_idx = next(splitter.split(df, groups=df[group_col]))
    return df.iloc[train_idx], df.iloc[test_idx]
```

1. **Line 9.** test_size is the share of groups, not rows. The seed makes the split reproducible.
2. **Line 10.** Every row of a respondent lands on the same side.

Tune on validation, measure once on test, and split by the unit that will be new in production.

Recap card

- **Roles of train, validation and test** Fit, choose, measure once.
- **Rows repeat per respondent** Split by group (GroupKFold, GroupShuffleSplit), or the model learns people.
- **Group leakage demonstration** Random CV 0.995, GroupKFold 0.468 on a label unrelated to the features.
- **Rare class** stratify=y keeps the class share in each set.

Practice

Exercise 1. Multiple choice (level 1)

You tried 40 model settings and picked the one with the best test score. What is that test score now?

- a) An unbiased estimate of performance on new data.
- b) An optimistic estimate, because the test set chose the model.
- c) A pessimistic estimate.
- d) The same as the training score.

Why?

- r1) Picking the best of 40 on the same data selects for luck on that data.
- r2) Test sets always overstate performance.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit

df = pd.DataFrame({"rid": ["a", "a", "b", "b", "c", "c", "d", "d"], "x":
   range(8)})
splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=0)
train_idx, test_idx = next(splitter.split(df, groups=df["rid"]))
print(len(test_idx), df.iloc[test_idx]["rid"].nunique())
```

Exercise 3. Fill the gap (level 2)

Keep the rare positive class at the same rate in train and test.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
   _____(1)=y, _____(2)=42)
```

Exercise 4. Write the function (level 3)

Write

`split_by_group(df, group_col, test_size, seed) -> tuple[pd.DataFrame, pd.DataFrame]` with `GroupShuffleSplit`, so that no value of `group_col` appears in both returned frames. The same seed must give the same split.

Your code must pass these checks:

- No respondent may appear in both train and test.
- Every row must land in one set, and 20% of the 20 respondents go to test.
- The same seed must give the same split.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Split rows so no group appears in both train and test.
def split_by_group(
    df: pd.DataFrame,
    group_col: str,
    test_size: float,
    seed: int,
) -> tuple[pd.DataFrame, pd.DataFrame]: ...
```

Starter code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split


def split_by_group(df, group_col, test_size, seed):
    return train_test_split(df, test_size=test_size, random_state=seed)
```

Exercise 5. Architecture design (level 4)

Choose the split for each project.

Options: Group split by member, Time split, Stratified random split.

1. Predict drop-out for newly recruited panel members; data has 6 waves per member
2. Fraud flag at 0.8% prevalence, one row per independent transaction, no time drift
3. Forecast next month's store sales

Sources

- Random 5-fold CV gave 0.995 and GroupKFold gave 0.468 on 200 simulated respondents with 5 waves each and a person-level random label. (verified: Run with scikit-learn 1.8.0 and 1.9.1, RandomForestClassifier(n_estimators=100, random_state=0), in this repository.)

B11-L2 Pipeline and ColumnTransformer

Concept

Lesson B10-L2 fitted preprocessing on training rows by hand. scikit-learn's `Pipeline` does it for you, every time, including inside each cross-validation fold. Examples use scikit-learn 1.8.

A `Pipeline` chains steps. Calling `fit` fits each step on the training rows and passes its output to the next. Calling `predict` runs the fitted steps on new rows **without refitting**. Cross-validation, grid search and `cross_val_score` all refit the whole pipeline per fold, so no fold's test rows leak into its preprocessing.

A `ColumnTransformer` applies different preprocessing to different columns:

```
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
    StandardScaler()), NUMERIC),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")),
    CATEGORICAL),
])
model = Pipeline([("prep", prep), ("clf",
    LogisticRegression(max_iter=1000))])
```

Three defaults to know:

- `OneHotEncoder(handle_unknown="error")` is the default. A region that did not appear in training raises `ValueError` at prediction time. Set `handle_unknown="ignore"` to encode it as all zeros.
- `ColumnTransformer(remainder="drop")` is the default. Columns you do not list are **silently dropped**. List every column you mean to use, or set `remainder="passthrough"` on purpose.
- Output feature names come from `get_feature_names_out()`, prefixed with the step name: `num__age`, `cat__region_n`.

Keep the column lists in config (lesson A1-L3), and have one function build the pipeline. The same fitted object is what you save, test and deploy, so preprocessing in production is exactly what was validated.

Why it matters

A lead-scoring model was trained in a notebook. Preprocessing ran in a separate cell: median imputation and scaling by hand. The deployment team rewrote the preprocessing from the notebook for production and imputed with the **mean**. Income was skewed, so the mean was 40% above the median. Scores for every lead with missing income shifted, and the sales team's call list changed without anyone noticing.

A single fitted `Pipeline`, saved and deployed as one object, leaves no second copy of the preprocessing to get wrong.

Worked example: weak code

```
X["age"] = X["age"].fillna(X["age"].median())
```

```
❷ X = pd.get_dummies(X, columns=["region"])
  3 scores = cross_val_score(LogisticRegression(), X, y, cv=5)
```

1. **Line 1.** Imputation on all rows, before CV: every fold's test rows shape the median.
2. **Line 2.** Dummy columns depend on the data seen. A new region in production changes the columns.

Worked example: strong code

```
  1 from sklearn.compose import ColumnTransformer
  2 from sklearn.impute import SimpleImputer
  3 from sklearn.linear_model import LogisticRegression
  4 from sklearn.pipeline import Pipeline, make_pipeline
  5 from sklearn.preprocessing import OneHotEncoder, StandardScaler
  6
  7
  8 def build_pipeline(numeric: list[str], categorical: list[str]) ->
      Pipeline:
  9     """Preprocessing and model as one object, fitted per fold by CV."""
 10     prep = ColumnTransformer(
 11         [
 12             ("num", make_pipeline(SimpleImputer(strategy="median"),
      StandardScaler()), numeric),
 13             (
 14                 "cat",
❶                  make_pipeline(SimpleImputer(strategy="most_frequent"),
      OneHotEncoder(handle_unknown="ignore")),
 16                 categorical,
 17             ),
 18         ]
 19     )
❷     return Pipeline([("prep", prep), ("clf",
      LogisticRegression(max_iter=1000))])
```

1. **Line 15.** An unseen category becomes all zeros instead of an error.
2. **Line 20.** One object to cross-validate, save and deploy.

Put every learning step in one Pipeline, split columns with a ColumnTransformer, and deploy the fitted object itself.

Recap card

- **Why Pipeline** Every learning step refits per CV fold, and one fitted object goes to production.
- **OneHotEncoder default for unseen categories** handle_unknown="error": raises ValueError. Use "ignore" for production.
- **ColumnTransformer default for unlisted columns** remainder="drop": silently dropped.

- **Feature names after preprocessing** get_feature_names_out(), prefixed with step names such as num__age.

Practice

Exercise 1. Multiple choice (level 1)

Why does putting a scaler inside a Pipeline prevent leakage in `cross_val_score`?

- a) The Pipeline refits the scaler on each fold's training rows only.
- b) Pipelines do not scale test data.
- c) The scaler is fitted once on all rows, but faster.
- d) Pipelines remove outliers automatically.

Why?

- r1) cross_val_score clones and fits the whole pipeline per fold.
- r2) Scalers inside pipelines have no parameters.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder().fit(pd.DataFrame({"region": ["n", "s"]}))
try:
    enc.transform(pd.DataFrame({"region": ["e"]}))
    print("encoded")
except ValueError:
    print("ValueError")
```

Exercise 3. Fill the gap (level 2)

Keep unlisted columns instead of dropping them, and encode unseen categories as zeros.

```
prep = ColumnTransformer(
    [("cat", OneHotEncoder(handle_unknown="_____(1)"), ["region"])],
    remainder="_____(2)",
)
```

Exercise 4. Write the function (level 3)

Write `build_pipeline(numeric, categorical) -> Pipeline` with two named steps:

- `"prep"`: a `ColumnTransformer` with `"num"` (median `SimpleImputer`, then `StandardScaler`) on `numeric`, and `"cat"` (most-frequent `SimpleImputer`, then `OneHotEncoder(handle_unknown="ignore")`) on `categorical`.
- `"clf"`: `LogisticRegression(max_iter=1000)`.

Your code must pass these checks:

- The pipeline needs steps named prep and clf, in that order.
- Numeric columns need median imputation before scaling: the median of training age is 30.
- An unseen region must not raise. Use OneHotEncoder(handle_unknown="ignore").
- Name the ColumnTransformer parts num and cat.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Preprocessing and classifier as one estimator.
def build_pipeline(
    numeric: list[str],
    categorical: list[str],
) -> Pipeline: ...
```

Starter code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline


def build_pipeline(numeric, categorical):
    return Pipeline([("clf", LogisticRegression())])
```

Exercise 5. Spot the bug (level 4)

The model ignores `tenure`, although it is the strongest predictor in exploration. Tap the line that causes it, and fix the function.

```
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


def build(categorical: list[str]) -> Pipeline:
    """Encode categorical columns and fit a classifier on every
    feature."""
```

```
    prep = ColumnTransformer([("cat",
OneHotEncoder(handle_unknown="ignore"), categorical)])
    return Pipeline([("prep", prep), ("clf",
LogisticRegression(max_iter=1000))])
```

Write down the line number of the bug. Then write the fix.

Sources

- In scikit-learn 1.8, OneHotEncoder defaults to handle_unknown="error" and ColumnTransformer to remainder="drop". (verified: scikit-learn 1.8 API reference; signatures checked on 1.8.0 and 1.9.1 in this repository.)

B11-L3 Cross-validation and honest hyperparameter search

Concept

Cross-validation (CV) splits the training data into k folds, trains on k - 1 and scores on the remaining one, k times. Report the **mean and the spread** across folds. A mean AUC of 0.81 with fold scores from 0.70 to 0.90 is a less certain result than 0.81 with scores from 0.79 to 0.83.

`cross_validate` (scikit-learn 1.8) returns several metrics in one pass:

```
res = cross_validate(pipe, X, y, cv=StratifiedKFold(5, shuffle=True,
    random_state=7),
                     scoring=["roc_auc", "average_precision"])
res["test_roc_auc"].mean(), res["test_roc_auc"].std()
```

Hyperparameter search. `GridSearchCV` or `RandomizedSearchCV` tries settings and keeps the best by CV score. Inside a pipeline, name parameters as `step__param`: `{"clf__C": [0.01, 0.1, 1, 10]}`.

The catch: `best_score_` is the **best of many tries** on the same folds. It is optimistic, for the same reason as picking a model on the test set (lesson B11-L1). A simulation in this course's build: 20 datasets of pure noise, a k-nearest-neighbours grid of 60 settings. The mean `best_score_` was **0.598** accuracy. The honest estimate was **0.515**.

Get the honest estimate with **nested CV**: an outer CV loop scores the **whole search** as if it were one model.

```
search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
honest = cross_val_score(search, X, y, cv=outer, scoring="roc_auc")
```

Or keep a held-out test set that the search never sees, and score the refitted best model on it once.

Practical rules:

- Fix `random_state` in every splitter, so reruns compare like with like.
- Search on a log scale for regularisation strengths: 0.001, 0.01, 0.1, 1, 10.
- Prefer `RandomizedSearchCV` when the grid has more than about 50 combinations.
- Report the chosen settings, the honest score and the fold spread in the run log.

Why it matters

A team compared 300 configurations of a gradient-boosted model and reported the top `best_score_`, AUC 0.842, to the client as expected performance. The model went live and scored 0.80 over the first quarter. Nothing had drifted. The 0.842 had been the luckiest of 300 draws on the same five folds. A nested CV estimate of 0.80 would have set the right expectation from the start.

Worked example: weak code

```
search = GridSearchCV(pipe, {"clf__C": [0.001, 0.01, 0.1, 1, 10, 100]}, cv=5)
search.fit(X, y)
print("expected AUC", search.best_score_)
```

1. **Line 1.** Default scoring is accuracy for classifiers, not the AUC being reported.
2. **Line 3.** The best of several tries on the same folds: optimistic, and the wrong metric.

Worked example: strong code

```
import numpy as np
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline


def nested_cv_auc(pipe: Pipeline, X: np.ndarray, y: np.ndarray, grid: dict[str, list[float]], seed: int) -> float:
    """Honest AUC of the whole search: the search is refitted inside each outer fold."""
    inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
    outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed + 1)
    search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
    return float(cross_val_score(search, X, y, cv=outer, scoring="roc_auc").mean())
```

1. **Line 10.** The metric you will report is the metric you tune on.

2. **Line 11.** Outer folds never take part in choosing the settings.

Report CV mean and spread, tune with the reported metric, and estimate performance with nested CV or an untouched test set.

Recap card

- **Report CV results as** Mean and spread across folds, for the metric you care about.
- **Pipeline parameter names in a grid** step__param, such as clf__C. make_pipeline names steps after their class.
- **Is best_score_ an honest estimate?** No. Noise simulation: 0.598 best_score_ against 0.515 nested. Use nested CV or a held-out test set.
- **Nested CV in scikit-learn** cross_val_score(GridSearchCV(pipe, grid, cv=inner), X, y, cv=outer).

Practice

Exercise 1. Multiple choice (level 1)

After `GridSearchCV` over 80 settings, what is `best_score_`?

- a) An unbiased estimate of performance on new data
- b) The mean CV score of the best setting, optimistic because it was selected as the best
- c) The test-set score
- d) The training score of the refitted model

Why?

- r1) Selecting the maximum of many noisy scores favours settings that were lucky on these folds.
- r2) GridSearchCV always overfits.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

What you need to know.

Every step in a scikit-learn pipeline has a name, and each step's settings are addressed as `name__setting`.

- `Pipeline([("scale", StandardScaler()), ("clf", LogisticRegression())])` uses the names you give.
- `make_pipeline(...)` names each step after its class, in lower case.

`pipe.get_params()` returns a dict of every setting, keyed by those names:

```
pipe = make_pipeline(StandardScaler(), DecisionTreeClassifier())
[name for name, _ in pipe.steps]   # ['standardscaler',
  'decisiontreeclassifier']
"decisiontreeclassifier__max_depth" in pipe.get_params()   # True
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pipe = make_pipeline(StandardScaler(), LogisticRegression())
print("clf__C" in pipe.get_params(), "logisticregression__C" in
  pipe.get_params())
```

Exercise 3. Fill the gap (level 2)

Tune C of the step named clf, scoring by ROC AUC.

```
search = GridSearchCV(pipe, {"_____(1)": [0.01, 0.1, 1, 10]}, cv=inner,
  _____(2)="roc_auc")
```

Exercise 4. Write the function (level 3)

Write `nested_cv_auc(X, y, c_values, seed) -> float`. Build a pipeline with steps `"scale"` (`StandardScaler`) and `"clf"` (`LogisticRegression(max_iter=1000)`). Wrap it in `GridSearchCV` over `{"clf__C": c_values}` with an inner `StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)` and `scoring="roc_auc"`. Return the mean of `cross_val_score` of the search with an outer `StratifiedKFold(n_splits=5, shuffle=True, random_state=seed + 1)`, also scored by ROC AUC.

Your code must pass these checks:

- Use the inner and outer StratifiedKFold settings from the brief, scored by roc_auc.
- On noise the honest estimate must sit near 0.5 (0.5103 with these settings).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Nested CV estimate of ROC AUC for a tuned logistic regression.
def nested_cv_auc(
    X: np.ndarray,
    y: np.ndarray,
    c_values: list[float],
    seed: int,
) -> float: ...
```

Starter code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
```

```
3
4
5 def nested_cv_auc(X, y, c_values, seed):
6     search = GridSearchCV(LogisticRegression(), {"C": c_values},
    cv=5).fit(X, y)
7     return search.best_score_
```

Exercise 5. Architecture design (level 4)

You must tune a model and tell the client what AUC to expect. You have 8,000 labelled rows. Choose the plan.

Options: Hold out 20%, untouched until the end, Use all rows for the search.

1. Held-out test set
2. Search on the other 80%
3. Number given to the client

Sources

- Across 20 noise datasets with a 60-setting kNN grid, mean best_score_ was 0.598 and mean nested CV accuracy 0.515. (verified: Simulated with scikit-learn 1.9.1 in this repository.)
- cross_validate returns test_<metric> keys for each scorer. (verified: Run on scikit-learn 1.8.0 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B11-L1 Train, validation and test, split by the right unit

B11-L1 exercise 1

Hints. (1) What did the test set do here? (2) Is the best of 40 lucky? (3) Use validation for choosing.

Answer. b) An optimistic estimate, because the test set chose the model. Because: r1) Picking the best of 40 on the same data selects for luck on that data.

Choose with validation or CV. Keep the test set for one final measurement.

Common wrong answers.

- `a`: It would be, if the test set were used once. Selection on it adds optimism.
- `c`: Selection favours settings that got lucky on this data, so the bias is upwards.
- `d`: Test rows were not used for fitting, so it differs from the training score.
- `b|r2`: Right answer, wrong reason. Used once, a test set is unbiased.

B11-L1 exercise 2

Hints. (1) test_size counts groups, not rows. (2) How many respondents is 25% of 4? (3) Each has 2 rows.

Answer.

```
1 2 1
```

One of four respondents goes to test, with both rows. No respondent is split across sets.

Common wrong answers.

- `2 2`: A group split moves whole groups. 25% of 4 respondents is 1 respondent, with both of their rows.
- `1 1`: The test respondent brings every row: 2 rows.

B11-L1 exercise 3

Hints. (1) The argument that preserves class proportions. (2) The argument that fixes the seed. (3) `stratify` and `random_state`.

Answer.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    stratify=y, random_state=42)
```

`stratify=y` preserves the class balance. A fixed `random_state` makes the split repeatable.

Common wrong answers.

- g1 = `shuffle`: shuffle controls whether rows are shuffled. stratify keeps class shares.

B11-L1 exercise 4

Hints. (1) `GroupShuffleSplit(n_splits=1, test_size=..., random_state=seed)`. (2) `next(splitter.split(df, groups=df[group_col]))` gives index arrays. (3) Select with `df.iloc[...]`.

Answer.

```
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit


def split_by_group(
    df: pd.DataFrame, group_col: str, test_size: float, seed: int
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Split rows so no group appears in both train and test.

    Args:
        df: Rows to split.
        group_col: Column identifying the repeated unit, such as
respondent.
        test_size: Share of groups in the test set.
        seed: Random seed for reproducibility.

    Returns:
        (train, test).
    """
    splitter = GroupShuffleSplit(n_splits=1, test_size=test_size,
random_state=seed)
    train_idx, test_idx = next(splitter.split(df, groups=df[group_col]))
    return df.iloc[train_idx], df.iloc[test_idx]
```

Group splitting keeps each respondent on one side, so validation measures performance on new people, as production will.

B11-L1 exercise 5

Hints. (1) What will be new in production? (2) Are rows independent? (3) Is time part of the use?

Answer.

- Panel members will be new in production: group by member.
- Independent rows with a rare class: stratify on the target.
- Forecasting: train on the past, test on later months.

The split mirrors production: new people, rare classes kept, or the future.

Common wrong answers.

- panel = `stratified`: Members' waves would land on both sides. The model would recognise people, not patterns.
- fraud = `group`: Rows are independent here, so grouping adds nothing. The risk is a test set with too few frauds.
- forecast = `stratified`: A random split lets the model learn from months after the ones it is tested on.

B11-L2 Pipeline and ColumnTransformer

B11-L2 exercise 1

Hints. (1) What does cross_val_score do with the estimator in each fold? (2) Which rows does the scaler see when fitted? (3) Only that fold's training rows.

Answer. a) The Pipeline refits the scaler on each fold's training rows only. Because: r1) cross_val_score clones and fits the whole pipeline per fold.

Each fold gets a fresh copy of the pipeline, fitted on that fold's training rows. The test rows are only transformed.

Common wrong answers.

- `b`: Test rows are scaled, with the parameters learned from that fold's training rows.
- `c`: Fitting once on all rows is exactly the leak the pipeline avoids.
- `d`: Pipelines only run the steps you put in them.
- `a|r2`: Right answer, wrong reason. The scaler learns mean and scale, per fold.

B11-L2 exercise 2

Hints. (1) Was region e seen in fit? (2) What is the default handle_unknown? (3) "error".

Answer.

```
ValueError
```

The default raises on unseen categories. Use `handle_unknown="ignore"` for production inputs.

Common wrong answers.

- `encoded`: The default handle_unknown is "error". An unseen category raises ValueError.

B11-L2 exercise 3

Hints. (1) The handle_unknown value that does not raise. (2) The remainder value that keeps columns. (3) `ignore` and `passthrough`.

Answer.

```
prep = ColumnTransformer(
    [("cat", OneHotEncoder(handle_unknown="ignore"), ["region"])],
    remainder="passthrough",
)
```

`handle_unknown="ignore"` avoids errors on new categories. `remainder="passthrough"` keeps other columns. Use it on purpose, not by accident.

Common wrong answers.

- g2 = `drop`: drop is the default, and it discards every unlisted column.

B11-L2 exercise 4

Hints. (1) `make_pipeline(SimpleImputer(strategy="median"), StandardScaler())` for numbers. (2) `make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore"))` for categories. (3) `Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])`.

Answer.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


```

```
def build_pipeline(numeric: list[str], categorical: list[str]) -> Pipeline:
    """Preprocessing and classifier as one estimator.

    Args:
        numeric: Numeric feature columns.
        categorical: Categorical feature columns.

    Returns:
        An unfitted Pipeline with steps prep and clf.
    """
    prep = ColumnTransformer(
        [
            ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numeric),
            (
                "cat",
                make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore")),
                categorical,
            ),
        ]
    )
    return Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
```

One estimator holds every learned step. Cross-validation refits it per fold, and production runs exactly what was validated.

B11-L2 exercise 5

Hints. (1) Which columns does the ColumnTransformer list? (2) What happens to the others by default? (3) remainder="drop".

Answer. The bug is on line 9.

Fixed version:

```
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


def build(categorical: list[str]) -> Pipeline:
    """Encode categorical columns and fit a classifier on every feature."""
    prep = ColumnTransformer(
        [("cat", OneHotEncoder(handle_unknown="ignore"), categorical)],
        remainder="passthrough",
    )
    return Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
```

`ColumnTransformer` drops unlisted columns by default. Pass them through on purpose, or better, list and preprocess them explicitly.

Common wrong answers.

- Line 10: The classifier uses whatever prep outputs. prep never outputs tenure.

B11-L3 Cross-validation and honest hyperparameter search

B11-L3 exercise 1

Hints. (1) How was the best chosen? (2) On which data was it scored? (3) Selection on the same folds.

Answer. b) The mean CV score of the best setting, optimistic because it was selected as the best Because: r1) Selecting the maximum of many noisy scores favours settings that were lucky on these folds.

`best_score_` is the winner's CV score. Use nested CV or an untouched test set for the estimate you report.

Common wrong answers.

- `a`: It is the maximum over 80 candidates on the same folds, so it carries selection optimism.
- `c`: GridSearchCV does not use a separate test set.
- `d`: It is a CV score on held-out folds, not a training score.
- `b|r2`: Right answer, wrong reason. The search is fine. Its best score needs an outer check.

B11-L3 exercise 2

Hints. (1) How does make_pipeline name its steps? (2) Lower-case class names. (3) Parameter keys are step__param.

Answer.

```
1 False True
```

`make_pipeline` names steps `standardscaler` and `logisticregression`. Use `Pipeline([("clf", ...)])` when you want short, stable names for your grids.

Common wrong answers.

- `True True`: make_pipeline names steps after their class in lower case. There is no step called clf.

- `True False`: Step names come from make_pipeline: logisticregression, so the key is logisticregression__C.

B11-L3 exercise 3

Hints. (1) step__param. (2) The argument for the metric. (3) `clf__C` and `scoring`.

Answer.

```
search = GridSearchCV(pipe, {"clf__C": [0.01, 0.1, 1, 10]}, cv=inner,
    scoring="roc_auc")
```

Pipeline parameters are addressed as `step__param`. `scoring` sets the metric the search optimises.

Common wrong answers.

- g1 = `C`: Inside a Pipeline, prefix the parameter with the step name and two underscores: clf__C.
- g1 = `clf_C`: Two underscores separate the step name from the parameter: clf__C.

B11-L3 exercise 4

Hints. (1) Build the Pipeline, then the GridSearchCV around it. (2) Pass the search object, unfitted, to cross_val_score. (3) Both loops use scoring="roc_auc".

Answer.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold,
    cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


def nested_cv_auc(X: np.ndarray, y: np.ndarray, c_values: list[float],
    seed: int) -> float:
    """Nested CV estimate of ROC AUC for a tuned logistic regression.

    Args:
        X: Features.
        y: Binary target.
        c_values: Candidate regularisation strengths.
        seed: Base seed for the inner and outer splitters.

    Returns:
        Mean outer-fold ROC AUC of the whole search.
    """
```

```
    pipe = Pipeline([("scale", StandardScaler()), ("clf",
LogisticRegression(max_iter=1000))])
    inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=seed)
    outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed +
1)
    search = GridSearchCV(pipe, {"clf__C": c_values}, cv=inner,
scoring="roc_auc")
    return float(cross_val_score(search, X, y, cv=outer,
scoring="roc_auc").mean())
```

The outer loop treats the search as one model. Its folds never influence which C is chosen, so the mean is an honest estimate.

B11-L3 exercise 5

Hints. (1) Which data never influences a choice? (2) Which metric is tuned? (3) Which number is honest?

Answer.

- Hold out a test set at the start.
- Search on the rest with CV, a fixed seed and the reported metric.
- Report one test-set score, plus the fold spread from CV as a guide to uncertainty.

Keep one set of data out of every decision, and report its score once.

Common wrong answers.

- search = `best_test`: Choosing on the test set makes its score optimistic.
- report = `best_score`: best_score_ is the best of many tries on the same folds.
- holdout = `use_all`: Without a holdout, use nested CV for the estimate. Otherwise the number is optimistic.